



Sums of Products for Mutually Recursive Datatypes

The Appropriationist's View on Generic Programming

Victor Cacciari Miraldo
Information and Computing Sciences
Utrecht University
Utrecht, The Netherlands
V.CacciariMiraldo@uu.nl

Alejandro Serrano
Information and Computing Sciences
Utrecht University
Utrecht, The Netherlands
A.SerranoMena@uu.nl

Abstract

Generic programming for mutually recursive families of datatypes is hard. On the other hand, most interesting abstract syntax trees are described by a mutually recursive family of datatypes. We could give up on using that mutually recursive structure, but then we lose the ability to use those generic operations which take advantage of that same structure. We present a new approach to generic programming that uses modern Haskell features to handle mutually recursive families with explicit *sum-of-products* structure. This additional structure allows us to remove much of the complexity previously associated with generic programming over these types.

CCS Concepts • Software and its engineering → Functional languages; Data types and structures;

Keywords Generic Programming, Datatype, Haskell

ACM Reference Format:

Victor Cacciari Miraldo and Alejandro Serrano. 2018. Sums of Products for Mutually Recursive Datatypes: The Appropriationist's View on Generic Programming. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Type-Driven Development (TyDe '18), September 27, 2018, St. Louis, MO, USA*. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3240719.3241786>

1 Introduction

(Datatype-)generic programming provides a mechanism to write functions by induction on the structure of algebraic datatypes [7]. A well-known example is the **deriving** mechanism in Haskell, which frees the programmer from writing repetitive functions such as equality [14]. A vast range of

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. TyDe '18, September 27, 2018, St. Louis, MO, USA
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-5825-5/18/09...\$15.00
<https://doi.org/10.1145/3240719.3241786>

approaches are available as preprocessors, language extensions, or libraries for Haskell [13, 19]. In Figure 1 we outline the main design differences between a few of those libraries.

The core idea underlying generic programming is the fact that a great number of datatypes can be described in a uniform fashion. Consider the following datatype representing binary trees with data stored in their leaves:

```
data Bin a = Leaf a | Bin (Bin a) (Bin a)
```

A value of type *Bin a* consists of a choice between two constructors. For the first choice, it also contains a value of type *a* whereas for the second it contains two subtrees as children. This means that the *Bin a* type is isomorphic to *Either a (Bin a, Bin a)*.

Different libraries differ on how they define their underlying generic descriptions. For example, `GHC.Generics` [12] defines the representation of *Bin* as the following datatype:

```
Rep (Bin a) = K1 R a :+: (K1 R (Bin a) :*: K1 R (Bin a))
```

which is a direct translation of *Either a (Bin a, Bin a)*, but using the combinators provided by `GHC.Generics`, namely `:+:` and `:*:`. In addition, we need two conversion functions *from* :: *a* → *Rep a* and *to* :: *Rep a* → *a* which form an isomorphism between *Bin a* and *Rep (Bin a)*. All this information is tied to the original datatype using a type class:

```
class Generic a where
  type Rep a :: *
  from :: a      → Rep a
  to   :: Rep a → a
```

Most generic programming libraries follow a similar pattern of defining the *description* of a datatype in the provided uniform language by some type level information, and two functions witnessing an isomorphism. A important feature of such library is how this description is encoded and which are the primitive operations for constructing such encodings, as we shall explore in Section 1.2. Some libraries, mainly deriving from the SYB approach [10, 16], use the *Data* and *Typeable* type classes instead of static type level information to provide generic functionality. These are a completely different strand of work from ours.

Figure 1 shows the main libraries relying on type level representations. In the *pattern functor* approach we have `GHC.Generics` [12], being the most commonly used one, that effectively replaced regular [17]. The former does not

	Pattern Functors	Codes
No Explicit Recursion	GHC.Generics	generics-sop
Simple Recursion	regular	generics-mrsop
Mutual Recursion	multirec	

Figure 1. Spectrum of static generic programming libraries

account for recursion explicitly, allowing only for a *shallow* representation, whereas the later allows for both *deep* and *shallow* representations by maintaining information about the recursive occurrences of a type. Maintaining this information is central to some generic functions, such as the generic *map* and *Zipper*, for instance. Oftentimes though, one actually needs more than just one recursive type, justifying the need to *multirec* [27].

These libraries are too permissive though, for instance, `K1 R Int :: Maybe` is a perfectly valid GHC.Generics *pattern functor* but will break generic functions, i.e., *Maybe* is not a combinator. The way to fix this is to ensure that the *pattern functors* abide by a certain format, by defining them by induction on some *code* that can be inspected and matched on. This is the approach of *generics-sop* [4]. The more restrictive code approach allows one to write concise, combinator-based, generic programs. The novelty in our work is in the intersection of both the expressivity of *multirec*, allowing the encoding of mutually recursive families, with the convenience of the more modern *generics-sop* style. In fact, it is worth noting that neither of the aforementioned libraries *compete* with our work. We extend both in orthogonal directions, resulting in a new design altogether, that takes advantage of some modern Haskell extensions that the authors of the previous work could not enjoy.

1.1 Contributions

In this paper we make the following contributions:

- We extend the sum-of-products approach of de Vries and Löb [4] to care for recursion (Section 3), allowing for *shallow* and *deep* representations. We proceed generalizing even further to mutually recursive families of datatypes (Section 4).
- We illustrate the use of our library on familiar examples such as equality, α -equivalence (Section 5.2) and the zipper (Section 5), illustrating how it subsumes the features of the previous approaches.
- We provide Template Haskell functionality to derive all the boilerplate code needed to use our library (in Appendix B, due to space restrictions). The novelty lies in our handling of instantiated type constructors.

We have packaged our results as a Haskell library. This library, *generics-mrsop*, fills the hole in Figure 1 for a code-based approach with support for mutual recursion.

1.2 Design Space

The availability of several libraries for generic programming witnesses the fact that there are trade-offs between expressivity, ease of use, and underlying techniques in the design of such a library. In this section we describe some of these trade-offs, especially those to consider when using the static approach.

Explicit Recursion. There are two ways to define the representation of values. Those that have information about which fields of the constructors of the datatype in question are recursive versus those that do not.

If we do not mark recursion explicitly, *shallow* encodings are our sole option, where only one layer of the value is turned into a generic form by a call to *from*. This is the kind of representation we get from GHC.Generics, among others. The other side of the spectrum would be the *deep* representation, in which the entire value is turned into the representation that the generic library provides in one go.

Marking the recursion explicitly, like in *regular* [17], allows one to choose between *shallow* and *deep* encodings at will. These representations are usually more involved as they need an extra mechanism to represent recursion. In the *Bin* example, the description of the *Bin* constructor changes from “this constructor has two fields of the *Bin a* type” to “this constructor has two fields in which you recurse”. Therefore, a *deep* encoding requires some explicit *least fixpoint* combinator – usually called *Fix* in Haskell.

Depending on the use case, a shallow representation might be more efficient if only part of the value needs to be inspected. On the other hand, deep representations are sometimes easier to use, since the conversion is performed in one go, and afterwards one only has to work with the constructs from the generic library.

The fact that we mark explicitly when recursion takes place in a datatype gives some additional insight into the description. Some functions really need the information about which fields of a constructor are recursive and which are not, like the generic *map* and the generic *Zipper* – we describe the latter in Section 5. This additional power has also been used to define regular expressions over Haskell datatypes [20].

Sum of Products Most generic programming libraries build their type level descriptions out of three basic combinators: (1) *constants*, which indicate a type is atomic and should not be expanded further; (2) *products* (usually written as `:::`) which are used to build tuples; and (3) *sums* (usually written as `:+:`) which encode the choice between constructors. *Rep (Bin a)* above is expressed in this form. Note, however, that there is no restriction on *how* these can be combined.

In practice, one can always use a sum of products to represent a datatype – a sum to express the choice of constructor, and within each constructor a product to declare which fields you have. The *generic-sop* library [4] explicitly uses a list of lists of types, the outer one representing the sum and each

inner one thought of as products. The `'` sign in the code below marks the list as operating at the type level, as opposed to term-level lists which exist at run-time. This is an example of Haskell's *datatype* promotion [28].

```
Codesop (Bin a) = '['[a], '[Bin a, Bin a]]
```

The shape of this description follows more closely the shape of Haskell datatypes, and make it easier to implement generic functionality.

Note how the *codes* are different than the *representation*. The latter being defined by induction on the former. This is quite a subtle point and it is common to see both terms being used interchangeably. Here, the *representation* is mapping the *codes*, of kind `'['[*]]`, into `*`. The *code* can be seen as the format that the *representation* must adhere to. Previously, in the pattern functor approach, the *representation* was not guaranteed to have a certain structure. The expressivity of the language of *codes* is proportional to the expressivity of the combinators the library can provide.

Mutually recursive datatypes. We have described several axes taken by different approaches to generic programming in Haskell. Unfortunately, most of the approaches restrict themselves to *regular* types, in which recursion always goes into the *same* datatype, which is the one being defined. Sometimes one would like to have the mutually recursive structure handy, though. The syntax of many programming languages, for instance, is expressed naturally using a mutually recursive family. Consider Haskell itself, one of the possibilities of an expression is to be a `do` block, while a `do` block itself is composed by a list of statements which may include expressions.

```
data Expr = ... | Do [Stmt] | ...
data Stmt = Assign Var Expr | Let Var Expr
```

Another example is found in HTML and XML documents. They are better described by a Rose tree, which can be described by this family of datatypes:

```
data Rose a = Fork a [Rose a]
data [] a = [] | a:[a]
```

The mutual recursion becomes apparent once one instantiates *a* to some ground type, for instance:

```
data RoseI = Fork Int ListI
data ListI = Nil | RoseI:ListI
```

The `multirec` library [27] is a generalization of `regular` which handles mutually recursive families using this very technique. The mutual recursion is central to some applications such as generic diffing [15] of abstract syntax trees.

The motivation of our work stems from the desire of having the concise structure that *codes* give to the *representations*, together with the information for recursive positions in a mutually recursive setting.

Deriving the representation. Generic programming alleviates the problem of repetitively writing operations such as equality or pretty-printing, which depend on the structure of the datatype. But in order to do so, they still require the programmer to figure out the right description and write conversion functions *from* and *to* that type. This is tedious, and also follows the shape of the type!

For that reason, most generic programming libraries also include some automatic way of generating this boilerplate code. `GHC.Generics` is embedded in the compiler; most others use Template Haskell [22], the meta-programming facility found in `GHC`. In the former case, programmers write:

```
data Bin a = ... deriving Generic
```

to open the doors to generic functionality.

There is an interesting problem that arises when we have mutually recursive datatypes and want to automatically generate descriptions. The definition of `Rose a` above uses the list type, but not simply `[a]` for any element type *a*, but the specific instance `[Rose a]`. This means that the procedure to derive the code must take this fact into account. Shallow descriptions do not suffer too much from this problem. For deep approaches, though, how to solve this problem is key to derive a useful description of the datatype.

2 Background

Before diving head first into our generic programming framework, let us take a tour of the existing generic programming libraries. For that, we will be looking at a generic *size* function from a few different angles, illustrating how different techniques relate and the nuances between them. This will let us gradually build up to our framework, that borrows pieces of each of the different approaches, and combines them without compromise.

2.1 GHC Generics

Since version 7.2, `GHC` supports some off the shelf generic programming using `GHC.Generics` [12], which exposes the *pattern functor* of a datatype. This allows one to define a function for a datatype by induction on the structure of its (shallow) representation using *pattern functors*.

These *pattern functors* are parametrized versions of tuples, sum types (*Either* in Haskell lingo), and unit, empty and constant functors. These provide a unified view over data: the generic *representation* of values. The values of a suitable type *a* are translated to this representation by means of the function $from_{gen} :: a \rightarrow Rep_{gen} a$. Note that the subscripts are there solely to disambiguate names that appear in many libraries. Hence, $from_{gen}$ is, in fact, the *from* in module `GHC.Generics`.

Defining a generic function is done in two steps. First, we define a class that exposes the function for arbitrary types, in our case, *size*, which we implement for any type via *gsize*:


```

size (Bin (Leaf 1) (Leaf 2))
= gsize (fromgen (Bin (Leaf 1) (Leaf 2)))
= gsize (R1 (K1 (Leaf 1) :* K1 (Leaf 2)))
= gsize (K1 (Leaf 1)) + gsize (K1 (Leaf 2))
 $\dagger$ 
= size (Leaf 1) + size (Leaf 2)
= gsize (fromgen (Leaf 1)) + gsize (fromgen (Leaf 2))
= gsize (L1 (K1 1)) + gsize (L1 (K1 2))
= size (1 :: Int) + size (2 :: Int)

```

Figure 2. Reduction of `size (Bin (Leaf 1) (Leaf 2))`

```

class Size (a :: *) where
  size :: a → Int

instance (Size a) ⇒ Size (Bin a) where
  size = gsize ∘ fromgen

```

Next we define the `gsz` function that operates on the level of the representation of datatypes. We have to use another class and the instance mechanism to encode a definition by induction on representations:

```

class GSize (rep :: * → *) where
  gsize :: rep x → Int

instance (GSize f, GSize g) ⇒ GSize (f :* g) where
  gsize (f :* g) = gsize f + gsize g

instance (GSize f, GSize g) ⇒ GSize (f :+: g) where
  gsize (L1 f) = gsize f
  gsize (R1 g) = gsize g

```

We still have to handle the cases where we might have an arbitrary type in a position, modeled by the constant functor `K1`. We require an instance of `Size` so we can successfully tie the recursive knot.

```

instance (Size x) ⇒ GSize (K1 R x) where
  gsize (K1 x) = size x

```

To finish the description of the generic `size`, we also need instances for the `unit`, `void` and `metadata` pattern functors, called `U1`, `V1`, and `M1` respectively. Their `GSize` is rather uninteresting, so we omit them for the sake of conciseness.

This technique of *mutually recursive classes* is quite specific to GHC. Generics flavor of generic programming. Figure 2 illustrates how the compiler goes about choosing instances for computing `size (Bin (Leaf 1) (Leaf 2))`. In the end, we just need an instance for `Size Int` to compute the final result. Literals of type `Int` illustrate what we call *opaque types*: those types that constitute the base of the universe and are *opaque* to the representation language.

One interesting aspect we should note here is the clearly *shallow* encoding that `from` provides. That is, we only represent *one layer* at a time. For example, take the step marked as ($\dagger$) in Figure 2: after unwrapping the calculation of the first *layer*, we are back to having to calculate `size` for `Bin Int`, not their generic representation.

Upon reflecting on the generic `size` function above, we see a number of issues. Most notably is the amount of boilerplate

to achieve a conceptually simple task: sum up all the sizes of the fields of whichever constructors make up the value. This is a direct consequence of not having access to the *sum-of-products* structure that Haskell's `data` declarations follow. A second issue is that the generic representation does not carry any information about the recursive structure of the type. The regular [17] library addresses this issue by having a specific *pattern functor* for recursive positions.

Perhaps even more subtle, but also more worrying, is that we have no guarantees that the `Repgen a` of a type `a` will be defined using only the supported *pattern functors*. Fixing this would require one to pin down a single language for representations, that is, the *code* of the datatype. Besides correctness issues, having *codes* greatly improves the definition of *ad-hoc* generic combinators. Every generic function has to follow the *mutually recursive classes* technique we shown.

2.2 Explicit Sums of Products

We will now examine the approach used by de Vries and Löb [4]. The main difference is in the introduction of *Codes*, that limit the structure of representations.

Had we had access to a representation of the *sum-of-products* structure of `Bin`, we could have defined our `gsz` function following an informal description: sum up the sizes of the fields inside a value, ignoring the constructor.

Unlike GHC. Generics, the representation of values is defined by induction on the *code* of a datatype, this *code* is a type level list of lists of kind `*`, whose semantics is consonant to a formula in disjunctive normal form. The outer list is interpreted as a sum and each of the inner lists as a product. This section provides an overview of generic-sop as required to understand our techniques, we refer the reader to the original paper [4] for a more comprehensive explanation.

Using a *sum-of-products* approach one could write the `gsz` function as easily as:

```

gsz :: (Genericsop a) ⇒ a → Int
gsz = sum ∘ elim (map size) ∘ fromsop

```

Ignoring the details of `gsz` for a moment, let us focus just on its high level structure. Remembering that `from` now returns a *sum-of-products* view over the data, we are using an eliminator, `elim`, to apply a function to the fields of the constructor used to create a value of type `a`. This eliminator then applies `map size` to the fields of the constructor, returning something akin to a `[Int]`. We then `sum` them up to obtain the final size.

Codes consist of a type level list of lists. The outer list represents the constructors of a type, and will be interpreted as a sum, whereas the inner lists are interpreted as the fields of the respective constructors, interpreted as products.

```

type family Codesop (a :: *) :: '['[*]]
type instance Codesop (Bin a) = '['[a], '[Bin a, Bin a]]

```

The *representation* is then defined by induction on Code_{sop} by the means of generalized n -ary sums, NS , and n -ary products, NP . With a slight abuse of notation, one can view NS and NP through the lens of the following type isomorphisms:

$$\begin{aligned}\text{NS } f [k_1, k_2, \dots] &\equiv f k_1 :+ (f k_2 :+ \dots) \\ \text{NP } f [k_1, k_2, \dots] &\equiv f k_1 :* (f k_2 :* \dots)\end{aligned}$$

We could then define Rep_{sop} to be $\text{NS } (\text{NP } (K1 R))$, echoing the isomorphisms above, where $\text{data } K1 R a = K1 a$ is borrowed from `GHC.Generics`. Note that we already need the parameter f to pass NP to NS here. This is exactly the representation we get from `GHC.Generics`.

$$\begin{aligned}\text{Rep}_{\text{sop}} (\text{Bin } a) &\equiv \text{NS } (\text{NP } (K1 R)) (\text{Code}_{\text{sop}} (\text{Bin } a)) \\ &\equiv K1 R a :+ (K1 R (\text{Bin } a) :* K1 R (\text{Bin } a)) \\ &\equiv \text{Rep}_{\text{gen}} (\text{Bin } a)\end{aligned}$$

It makes no sense to go through all the trouble of adding the explicit *sums-of-products* structure to forget this information in the representation. Instead of piggybacking on *pattern functors*, we define NS and NP from scratch using *GADTs* [26]. By pattern matching on the values of NS and NP we inform the type checker of the structure of Code_{sop} .

```
data NS :: (k → *) → [k] → * where
  Here :: f k → NS f (k' : ks)
  There :: NS f ks → NS f (k' : ks)

data NP :: (k → *) → [k] → * where
  NP0 :: NP f '[]
  (×) :: f x → NP f xs → NP f (x' : xs)
```

Finally, since our atoms are of kind $*$, we can use the identity functor, I , to interpret those and define the final representation of values of a type a under the *SOP* view:

```
type Repsop a = NS (NP I) (Codesop a)
newtype I (a :: *) = I { unI :: a }
```

To support the claim that one can define general combinators for working with these representations, let us look at *elim* and *map*, used to implement the *gsize* function in the beginning of the section. The *elim* function just drops the constructor index and applies f , whereas the *map* applies f to all elements of a product.

```
elim :: (∀ k . f k → a) → NS f ks → a
elim f (Here x) = f x
elim f (There x) = elim f x

map :: (∀ k . f k → a) → NP f ks → [a]
map f NP0 = []
map f (x × xs) = f x : map f xs
```

Reflecting on the current definition of *size*, especially in comparison to the `GHC.Generics` implementation of *size*, we see two improvements: (A) we need one fewer type class, namely *GSize*, and, (B) the definition is combinator-based. Considering that the generated *pattern functor* representation of a Haskell datatype will already be in a *sums-of-products*, we do not lose anything by enforcing this structure.

There are still downsides to this approach. A notable one is the need to carry constraints around: the actual *gsize* written with the `generics-sop` library and no sugar reads as follows.

```
gsize :: (Genericsop a, All2 Size (Codesop a)) ⇒ a → Int
gsize = sum ∘ hcollapse
      ∘ hmap (Proxy :: Proxy Size) (mapIK size) ∘ fromsop
```

Where *hcollapse* and *hmap* are analogous to the *elim* and *map* combinators we defined above. The *All2 Size (Code_{sop} a)* constraint tells the compiler that all of the types serving as atoms for $\text{Code}_{\text{sop}} a$ are an instance of *Size*. In our case, *All2 Size (Code_{sop} (Bin a))* expands to *(Size a, Size (Bin a))*. The *Size* constraint also has to be passed around with a *Proxy* for the eliminator of the n -ary sum. This is a direct consequence of a *shallow* encoding: since we only unfold one layer of recursion at a time, we have to carry proofs that the recursive arguments can also be translated to a generic representation. We can relieve this burden by recording, explicitly, which fields of a constructor are recursive or not.

3 Explicit Fix: Diving Deep and Shallow

In this section we will start to look at our approach, essentially combining the techniques from the regular and `generics-sop` libraries. Later we extend the constructions to handle mutually recursive families instead of simple recursion. As we discussed in the introduction, a fixpoint view over generic functionality is required to implement some functionality like the *Zipper* generically. In other words, we need an explicit description of which fields of a constructor are recursive and which are not.

Introducing information about the recursive positions in a type requires more expressive codes than in Section 2.2, where our *codes* were a list of lists of types, which could be anything. Instead, we will now have a list of lists of *Atom* to be our codes:

```
data Atom = I | KInt | ...
type family Codefix (a :: *) :: '[Atom]
type instance Codefix (Bin Int) = '[KInt], '[I, I]
```

Where I is used to mark the recursive positions and $KInt, \dots$ are codes for a predetermined selection of primitive types, which we refer to as *opaque types*. Favoring the simplicity of the presentation, we will stick with only hard coded *Int* as the only opaque type in the universe. Later on, in Section 4.1, we parametrize the whole development by the choice of opaque types.

We can no longer represent polymorphic types in this universe – the *codes* themselves are not polymorphic. Back in Section 2.2 we have defined $\text{Code}_{\text{sop}} (\text{Bin } a)$, and this would work for any a . This might seem like a disadvantage at first, but it is in fact the opposite. This allows us to provide a deep conversion for free and drops the need to carry constraints around. Beyond doubt one needs to have access to

the $Code_{sop}$ a when converting a Bin a to its deep representation. By specifying the types involved beforehand, we are able to get by without having to carry all of the constraints we needed, for instance, for $gsize$ at the end of Section 2.2. We can benefit the most from this in the simplicity of combinators we are able to write, as shown in Section 4.2.

Wrapping our to_{fix} and $from_{fix}$ isomorphism into a type class and writing the instance that witnesses that Bin Int has a $Code_{fix}$ is straightforward. We omit the to_{fix} function as it is the opposite of $from_{fix}$:

```
class Genericfix a where
  fromfix :: a → Repfix a a
  tofix   :: Repfix a a → a

instance Genericfix (Bin Int) where
  fromfix (Leaf x)
    = Rep ( Here (NAK x × NP0) )
  fromfix (Bin l r)
    = Rep ( There (Here (NAI l × NAI r × NP0)) )
```

In order to define Rep_{fix} we just need a way to map an $Atom$ into $*$. Since an atom can be either an opaque type, known statically, or some other type that will be used as a recursive position later on, we simply receive it as another parameter. The NA datatype relates an $Atom$ to its semantics:

```
data NA :: * → Atom → * where
  NAI :: x → NA x I
  NAK :: Int → NA x KInt

newtype Repfix a x
  = Rep { unRep :: NS (NP (NA x)) (Codefix a) }
```

It is an interesting exercise to implement the $Functor$ instance for $(Rep_{fix} a)$. We were only able to lift it to a functor by recording the information about the recursive positions. Otherwise, there would be no way to know where to apply f when defining $fmap f$.

Nevertheless, working directly with Rep_{fix} is hard – we need to pattern match on $There$ and $Here$, whereas we actually want to have the notion of *constructor* for the generic setting too! The main advantage of the *sum-of-products* structure is to allow a user to pattern match on generic representations just like they would on values of the original type, contrasting with $GHC.Generics$. One can precisely state that a value of a representation is composed by a choice of constructor and its respective product of fields by the $View$ type.

```
data Nat = Z | S Nat

data View :: [[Atom]] → * → * where
  Tag :: Constr n t → NP (NA x) (Lkup t n) → View t x
```

A value of $Constr$ n sum is a proof that n is a valid constructor for sum , stating that $n < length\ sum$. $Lkup$ performs list lookup at the type level. In order to improve type error messages, we generate a $TypeError$ whenever we reach a given index n that is out of bounds. Interestingly, our design guarantees that this case is never reached by $Constr$.

```
data Constr :: Nat → [k] → * where
  CZ :: Constr Z (x:xs)
  CS :: Constr n xs → Constr (S n) (x:xs)

type family Lkup (ls :: [k]) (n :: Nat) :: k where
  Lkup '[] _ = TypeError "Index out of bounds"
  Lkup (x:xs) 'Z = x
  Lkup (x:xs) ('S n) = Lkup xs n
```

Now we are able to easily pattern match and inject into and from generic values. Unfortunately, matching on Tag requires describing in full detail the shape of the generic value using the elements of $Constr$. Using pattern synonyms [18] we can define those patterns once and for all, and give them more descriptive names. For example, here are the synonyms describing the constructors Bin and $Leaf$.¹

```
pattern Leaffix x = Tag CZ (NAK x × NP0)
pattern Binfix l r = Tag (CS CZ) (NAI l × NAI r × NP0)
```

The functions that perform the pattern matching and injection are the inj and sop below.

```
inj :: View sop x → Repfix sop x
sop :: Repfix sop x → View sop x
```

The $View$ type and the hability to split a value into a choice of constructor and its fields is very handy for writing generic functions, as we can see in Section 5.2.

Having the core of the *sums-of-products* universe defined, we can turn our attention to writing the combinators that the programmer will use. These will be defined by induction on the $Code_{fix}$ instead of having to rely on instances, like in Section 2.1. For instance, let's look at $compos$, which applies a function f everywhere on the recursive structure.

```
compos :: (Genericfix a) ⇒ (a → a) → a → a
compos f = tofix ∘ fmap f ∘ fromfix
```

Although more interesting in the mutually recursive setting, Section 4, we can illustrate its use for traversing a tree and adding one to its leaves. This example is a bit convoluted, since one could get the same result by simply writing $fmap (+1) :: Bin Int → Bin Int$, but shows the intended usage of the $compos$ combinator just defined.

```
example :: Bin Int → Bin Int
example (Leaf n) = Leaf (n + 1)
example x        = compos example x
```

It is worth noting the *catch-all* case, allowing one to focus only on the interesting patterns and using a default implementation everywhere else.

Converting to a deep representation. The $from_{fix}$ function returns a shallow representation. But by constructing the least fixpoint of $Rep_{fix} a$ we can easily obtain the deep encoding for free, by simply recursively translating each layer of the shallow encoding.

¹Throughout this paper we use the syntax $\bar{C}$ to refer to the pattern describing a view for constructor C .

```

crush :: (Genericfix a)
  ⇒ (∀ x . Int → b) → ([b] → b)
  → a → b
crush k cat = crushFix ∘ deepFrom
where
  crushFix :: Fix (Repfix a) → b
  crushFix = cat ∘ elimNS (elimNP go) ∘ unFix
  go (NAI x) = crushFix x
  go (NAK i) = k i

```

Figure 3. Generic *crush* combinator

```

newtype Fix f = Fix { unFix :: f (Fix f) }
deepFrom :: (Genericfix a) ⇒ a → Fix (Repfix a)
deepFrom = Fix ∘ fmap deepFrom ∘ fromfix

```

So far, we handle the same class of types as the regular [17] library, but we are imposing the representation to follow a *sum-of-products* structure by the means of *Code_{fix}*. Those types are guaranteed to have an initial algebra, and indeed, the generic fold is defined as expected:

```

fold :: (Repfix a b → b) → Fix (Repfix a) → b
fold f = f ∘ fmap (fold f) ∘ unFix

```

Sometimes we actually want to consume a value and produce a single value, but do not need the full expressivity of *fold*. Instead, if we know how to consume the opaque types and combine those results, we can consume any *Generic_{fix}* type using *crush*, which is defined in fig. 3. The behavior of *crush* is defined by (1) how to turn atoms into the output type *b* – in this case we only have integer atoms, and thus we require an *Int* → *b* function – and (2) how to combine the values bubbling up from each member of a product. Finally, we come full circle to our running *gsize* example as it was promised in the introduction. This is noticeably the smallest implementation so far, and very straight to the point.

```

gsize :: (Genericfix a) ⇒ a → Int
gsize = crush (const 1) sum

```

Let us take a step back and reflect upon what we have achieved so far. We have combined the insight from the regular library of keeping track of recursive positions with the convenience of the *generics-sop* for enforcing a specific *normal form* on representations. By doing so, we were able to provide a *deep* encoding for free. This essentially frees us from the burden of maintaining complicated constraints needed for handling the types within the topmost constructor. The information about the recursive position allows us to write neat combinators like *crush* and *compos* together with a convenient *View* type for easy generic pattern matching. The only thing keeping us from handling real life applications is the limited form of recursion. When a user requires a generic programming library, chances are they need to traverse and consume mutually recursive structures.

4 Mutual Recursion

Conceptually, going from regular types (Section 3) to mutually recursive families is simple. We just need to be able to reference not only one type variable, but one for each element in the family. This is usually [2, 11] done by adding an index to the recursive positions that represents which member of the family we are recursing over. As a running example, we use the *rose tree* family from the introduction.

```

data Rose a = Fork a [Rose a]
data [] a = [] | a:[a]

```

The previously introduced *Code_{fix}* is not expressive enough to describe this datatype. In particular, when we try to write *Code_{fix} (Rose Int)*, there is no immediately recursive appearance of *Rose* itself, so we cannot use the atom *I* in that position. Furthermore *[Rose a]* is not an opaque type either, so we cannot use any of the other combinators provided by *Atom*. We would like to record information about *[Rose Int]* referring to itself via another datatype.

Our solution is to move from codes of datatypes to *codes for families of datatypes*. We no longer talk about *Code_{fix} (Rose Int)* or *Code_{fix} [Rose Int]* in isolation. Codes only make sense within a family, that is, a list of types. Hence, we talk about *Code_{mrec} '[Rose Int, [Rose Int]]*. That is, the codes of the two types in the family. Then we extend the language of *Atoms* by appending to *I* a natural number which specifies the member of the family to recurse into:

```

data Atom = I Nat | KInt | ...

```

The code of this recursive family of datatypes can finally be described as:

```

type FamRose = '[Rose Int, [Rose Int]]
type Codemrec FamRose = '[ '[KInt, I (S Z)]
                        , '[[], '[I Z, I (S Z)]]]

```

Let us have a closer look at the code for *Rose Int*, which appears in the first place in the list. There is only one constructor which has an *Int* field, represented by *KInt*, and another in which we recurse via the second member of our family (since lists are 0-indexed, we represent this by *S Z*). Similarly, the second constructor of *[Rose Int]* points back to both *Rose Int* using *I Z* and to *[Rose Int]* itself via *I (S Z)*.

Having settled on the definition of *Atom*, we now need to adapt *NA* to the new *Atoms*. In order to interpret any *Atom* into ***, we now need a way to interpret the different recursive positions. This information is given by an additional type parameter φ that maps natural numbers into types.

```

data NA :: (Nat → *) → Atom → * where
  NAI :: φ n → NA φ (I n)
  NAK :: Int → NA φ KInt

```

This additional φ naturally bubbles up to *Rep_{mrec}*.

```

type Repmrec (φ :: Nat → *) (c :: [[Atom]])
  = NS (NP (NA φ)) c

```

The only piece missing here is tying the recursive knot. If we

want our representation to describe a family of datatypes, the obvious choice for φn is to look up the type at index n in *FamRose*. In fact, we are simply performing a type level lookup in the family, so we can reuse the *Lkup* from Section 3.

In principle, this is enough to provide a ground representation for the family of types. Let *fam* be a family of types, like $[Rose\ Int, [Rose\ Int]]$, and *codes* the corresponding list of codes. Then the representation of the type at index *ix* in the list *fam* is given by:

$Rep_{mrec} (Lkup\ fam) (Lkup\ codes\ ix)$

This definition states that to obtain the representation of the type at index *ix*, we first lookup its code. Then, in the recursive positions we interpret each *I n* by looking up the type at that index in the original family. This gives us a *shallow* representation. As an example, below is the expansion for index 0 of the rose tree family. Note how it is isomorphic to the representation that GHC.Generics would have chosen for *Rose Int*:

```
Repmrec (Lkup FamRose) (Lkup (Codemrec FamRose) Z)
= Repmrec (Lkup FamRose) ' [KInt, I (S Z)]
= NS (NP (NA (Lkup FamRose))) ' [KInt, I (S Z)]
≡ K1 R Int :*: K1 R (Lkup FamRose (S Z))
= K1 R Int :*: K1 R [Rose Int]
= Repgen (Rose Int)
```

Unfortunately, Haskell only allows saturated, that is, fully-applied type families. Hence, we cannot partially apply *Lkup* like we did it in the example above. As a result, we need to introduce an intermediate datatype *El*,

data *El* :: [*] → Nat → * **where**
El :: Lkup fam ix → El fam ix

The representation of the family *fam* at index *ix* is thus given by $Rep_{mrec} (El\ fam) (Lkup\ codes\ ix)$. We only need to use *El* in the first argument, because that is the position in which we require partial application. The second position has *Lkup* already fully-applied, and can stay as is.

We still have to relate a family of types to their respective codes. As in other generic programming approaches, we want to make their relation explicit. The *Family* type class below realizes this relation, and introduces functions to perform the conversion between our representation and the actual types. Using *El* here spares us from using a proxy for *fam* in $from_{mrec}$ and to_{mrec} :

class *Family* (*fam* :: [*]) (*codes* :: [[*Atom*]]) **where**
 $from_{mrec} :: SNat\ ix$
 $\rightarrow El\ fam\ ix \rightarrow Rep_{mrec} (El\ fam) (Lkup\ codes\ ix)$
 $to_{mrec} :: SNat\ ix$
 $\rightarrow Rep_{mrec} (El\ fam) (Lkup\ codes\ ix) \rightarrow El\ fam\ ix$

One of the differences between other approaches and ours is that we do not use an associated type to define the *codes* for the family *fam*. One of the reasons to choose this path is that it alleviates the burden of writing the longer $Code_{mrec}\ fam$

every time we want to refer to *codes*. Furthermore, there are types like lists which appear in many different families, and in that case it makes sense to speak about a relation instead of a function. In any case, we can choose the other point of the design space by moving *codes* into an associated type or introduce a functional dependency $fam \rightarrow codes$.

Since now $from_{mrec}$ and to_{mrec} operate on families, we have to specify how to translate *each* of the members of the family back and forth the generic representation. This translation needs to know which is the index of the datatype we are converting between in each case, hence the additional *SNat ix* parameter. Pattern matching on this singleton [5] type informs the compiler about the shape of the *Nat* index. Its definition is:

data *SNat* (*n* :: Nat) **where**
SZ :: SNat 'Z
SS :: SNat n → SNat ('S n)

For example, in the case of our family of rose trees, $from_{mrec}$ has the following shape:

```
frommrec SZ (El (Fork x ch))
= Rep (Here (NAK x × NAI ch × NP0))
frommrec (SS SZ) (El [])
= Rep (Here NP0)
frommrec (SS SZ) (El (x:xs))
= Rep (There (Here (NAI x × NAI xs × NP0)))
```

By pattern matching on the index, the compiler knows which family member to expect as a second argument. This then allows the pattern matching on the *El* to typecheck.

The limitations of the Haskell type system lead us to introduce *El* as an intermediate datatype. Our $from_{mrec}$ function does not take a member of the family directly, but an *El*-wrapped one. However, to construct that value, *El* needs to know its parameters, which amounts to the family we are embedding our type into and the index in that family. Those values are not immediately obvious, but we can use Haskell's visible type application [6] to work around it. The *into* function injects a value into the corresponding *El*:

$into :: \forall\ fam\ ty\ ix. (ix \sim Idx\ ty\ fam, Lkup\ fam\ ix \sim ty)$
 $\Rightarrow ty \rightarrow El\ fam\ ix$
 $into = El$

where *Idx* is a closed type family implementing the inverse of *Lkup*, that is, obtaining the index of the type *ty* in the list *fam*. Using this function we can turn a $[Rose\ Int]$ into its generic representation by writing $from_{mrec} \circ into\ @FamRose$. The type application $@FamRose$ is responsible for fixing the mutually recursive family we are working with, which allows the type checker to reduce all the constraints and happily inject the element into *El*.

Deep representation. In Section 3 we have described a technique to derive deep representations from shallow representations. We can play a very similar trick here. The main difference is the definition of the least fixpoint combinator,

which receives an extra parameter of kind *Nat* indicating which *code* to use first:

```
newtype Fix (codes :: [[[Atom]]]) (ix :: Nat)
  = Fix { unFix :: Repmrec (Fix codes) (Lkup codes ix) }
```

Intuitively, since now we can recurse on different positions, we need to keep track of the representations for all those positions in the type. This is the job of the *codes* argument. Furthermore, our *Fix* does not represent a single datatype, but rather the *whole* family. Thus, we need each value to have an additional index to declare on which element of the family it is working on.

As in the previous section, we can obtain the deep representation by iteratively applying the shallow representation. Earlier we used *fmap* since the *Rep_{fix}* type was a functor. *Rep_{mrec}* on the other hand cannot be given a *Functor* instance, but we can still define a similar function *mapRec*,

```
mapRep :: (∀ ix . ϕ1 ix → ϕ2 ix)
  → Repmrec ϕ1 c → Repmrec ϕ2 c
```

This signature tells us that if we want to change the ϕ_1 argument in the representation, we need to provide a natural transformation from ϕ_1 to ϕ_2 , that is, a function which works over each possible index this ϕ_1 can take and does not change this index. This follows from ϕ_1 having kind *Nat* $\rightarrow *$.

```
deepFrom :: Family fam codes
  ⇒ El fam ix → Fix (Repmrec codes ix)
deepFrom = Fix ∘ mapRec deepFrom ∘ frommrec
```

Only well-formed representations are accepted. At first glance, it may seem like the *Atom* datatype gives too much freedom: its *I* constructor receives a natural number, but there is no apparent static check that this number refers to an actual member of the recursive family we are describing. For example, the list of codes `'[['[KInt, I (S (S Z))]]]` is accepted by the compiler although it does not represent any family of datatypes.

A direct solution to this problem is to introduce yet another index, this time in the *Atom* datatype, which specifies which indices are allowed. The *I* constructor is then refined to take not any natural number, but only those which lie in the range – this is usually known as *Fin* *n*.

```
data Atom (n :: Nat) = I (Fin n) | KInt | ...
```

The lack of dependent types makes this approach very hard, in Haskell. We would need to carry around the inhabitants *Fin* *n* and define functionality to manipulate them, which is more complex than what meets the eye. This could greatly hinder the usability of the library.

By looking a bit more closely, we find that we are not losing any type-safety by allowing codes which reference an arbitrary number of recursive positions. Users of our library are allowed to write the previous ill-defined code, but when trying to write *values* of the representation of that code, the *Lkup* function detects the out-of-bounds index, raising a type error and preventing the program from compiling.

4.1 Parametrized Opaque Types

Up to this point we have considered *Atom* to include a predetermined selection of *opaque types*, such as *Int*, each of them represented by one of the constructors other than *I*. This is far from ideal, for two conflicting reasons:

1. The choice of opaque types might be too narrow. For example, the user of our library may decide to use *ByteString* in their datatypes. Since that type is not covered by *Atom*, nor by our generic approach, this implies that *generics-mrsop* becomes useless to them.
2. The choice of opaque types might be too wide. If we try to encompass any possible situation, we end up with a huge *Atom* type. But for a specific use case, we might be interested only in *Ints* and *Floats*, so why bother ourselves with possibly ill-formed representations and pattern matches which should never be reached?

Our solution is to *parametrize* *Atom*, giving programmers the choice of opaque types:

```
data Atom kon = I Nat | K kon
```

For example, if we only want to deal with numeric opaque types, we can write:

```
data NumericK = KInt | KInteger | KFloat
type NumericAtom = Atom NumericK
```

The representation of codes must be updated to reflect the possibility of choosing different sets of opaque types. The *NA* datatype in this final implementation provides two constructors, one per constructor in *Atom*. The *NS* and *NP* datatypes do not require any change.

```
data NA :: (kon → *) → (Nat → *) → Atom kon → * where
  NAI :: ϕ n → NA κ ϕ (I n)
  NAK :: κ k → NA κ ϕ (K k)
type Repmrec (κ :: kon → *) (ϕ :: Nat → *) (c :: [[Atom kon]])
  = NS (NP (NA κ ϕ)) c
```

The *NA_K* constructor in *NA* makes use of an additional argument κ . The problem is that we are defining the code for the set of opaque types by a specific kind, such as *Numeric* above. On the other hand, values which appear in a field must have a type whose kind is $*$. Thus, we require a mapping from each of the codes to the actual opaque type they represent, this is exactly the *opaque type interpretation* κ . Here is the datatype interpreting *NumericK* into ground types:

```
data NumericI :: NumericK → * where
  IInt :: Int → NumericI KInt
  IInteger :: Integer → NumericI KInteger
  IFloat :: Float → NumericI KFloat
```

The last piece of our framework which has to be updated to support different sets of opaque types is the *Family* type class, as given in Figure 4. This type class provides an interesting use case for the new dependent features in Haskell;

both κ and *codes* are parametrized by an implicit argument *kon* which represents the set of opaque types.

We stress that the parametrization over opaque types does *not* mean that we can use only closed universes of opaque types. It is possible to provide an *open* representation by choosing $(*)$ – the whole kind of Haskell’s ground types – as argument to *Atom*. As a consequence, the interpretation ought to be of kind $* \rightarrow *$, as follows:

```
data Value :: * → * where
  Value :: t → Value t
```

In order to use $(*)$ as an argument to a type, we are required to enable the TypeInType language extension [23, 24].

4.2 Combinators

In the remainder of this section we wish to showcase a selection of particularly powerful combinators that are simple to define by exploiting the *sums-of-products* structure coupled with the mutual recursion information. Defining the same combinators in *multirec* would produce much more complicated code. In *GHC.Generics* these are even impossible to write due to the absence of recursion information.

For the sake of fostering intuition instead of worrying about notational overhead, we write values of $\text{Rep}_{\text{mrec}} \kappa \varphi c$ just like we would write normal Haskell values. They have the same *sums-of-products* structure anyway. Whenever a function is defined using the $\simeq$ symbol, $C \ x_1 \ \dots \ x_n$ will stand for a value of the corresponding $\text{Rep}_{\text{mrec}} \kappa \varphi c$, that is, *There* ($\dots$ (*Here* ($x_1 \times \dots \times x_n \times \text{NP0}$))). Since each of these $x_1 \ \dots \ x_n$ might be a recursive type or an opaque type, whenever we have two functions f_I and f_K in scope, $f \ x_j$ will denote the application of the correct function for recursive positions, f_I , or opaque types f_K . For example, here is the actual code of the function which maps over a *NA* structure:

```
bimapNA f_K f_I (NA_I i) = NA_I (f_I i)
bimapNA f_K f_I (NA_K k) = NA_K (f_K k)
```

which following this convention becomes:

```
bimapNA f_K f_I x ≐ f x
```

The first obvious combinator which we can write using the sum-of-products structure is *map*. Our $\text{Rep}_{\text{mrec}} \kappa \varphi c$ is no longer a regular functor, but a higher bifunctor. In other words, it requires two functions, one for mapping over opaque types and another for mapping over *I* positions.

```
bimapRep :: (∀ k . κ k → κ₂ k) → (∀ ix . ϕ₁ ix → ϕ₂ ix)
  → Rep_mrec κ₁ ϕ₁ c → Rep_mrec κ₂ ϕ₂ c
bimapRep f_K f_I (C x₁ ... x_n) ≐ C (f x₁) ... (f x_n)
```

More interesting than a map perhaps is a general eliminator. In order to destruct a $\text{Rep}_{\text{mrec}} \kappa \varphi c$ we need a way for eliminating every recursive position or opaque type inside the representation and a way of combining these results.

```
elimRep :: (∀ k . κ k → a) → (∀ ix . ϕ ix → a) → ([a] → b)
  → Rep_mrec κ ϕ c → b
elimRep f_K f_I cat (C x₁ ... x_n) ≐ cat [f x₁, ..., f x_n]
```

Being able to eliminate a representation is useful, but it becomes even more useful when we are able to combine the data in different values of the same representation with a *zip* like combinator. Our *zipRep* will attempt to put two values of a representation “side-by-side”, as long as they are constructed with the same injection into the *n*-ary sum, *NS*.

```
zipRep :: Rep_mrec κ₁ ϕ₁ c → Rep_mrec κ₂ ϕ₂ c
  → Maybe (Rep_mrec (κ₁ :*: κ₂) (ϕ₁ :*: ϕ₂) c)
zipRep (C x₁ ... x_n) (D y₁ ... y_m)
  | C ≐ D ≐ Just (C (x₁ :*: y₁) ... (x_n :*: y_n))
    -- if C == D, then also n == m!
  | otherwise ≐ Nothing
```

This definition *zipRep* can be translated to work with an arbitrary (*Alternative f*) instead of *Maybe*. The *compos* combinator, already introduced in Section 3, shows up in a yet more expressive form. We are now able to change every subtree of whatever type we choose inside an arbitrary value of the mutually recursive family in question.

```
compos :: (∀ iy . El fam iy → El fam iy)
  → El fam ix → El fam ix
compos f = to_mrec ∘ bimapRep id f ∘ from_mrec
```

Defining these combinators in *multirec* is not impossible, but involves a much bigger effort. Everything has to be implemented by the means of type classes and each supported combinator must have one instance.

It is worth noting that although we presented pure versions of these combinators, *generics-mrsop* defines monadic variants of these and suffixes them with a *M*, following the standard Haskell naming convention. We will need these monadic combinators in Section 5.2.

5 Examples

In this section we present two applications of our generic programming approach, namely equality and α -equivalence. Our goal is to show that our approach is at least as powerful as any other comparable library, but brings in the union of their advantages. Even though some examples use a single recursive datatype for the sake of conciseness, those can be readily generalized to mutually recursive families. Another common benchmark for the power of a generic library, *zippers*, is described in Appendix A due to lack of space.

There are many other applications for generic programming which greatly benefit from supporting mutual recursion, if not requiring it. One great source of examples consists of operations on abstract syntax trees of realistic languages, such as generic diffing [15] or pretty-printing [12].

5.1 Equality

As usually done in generic programming papers, we should define generic equality in our own framework. In fact, with

```

class Family (κ :: kon → *) (fam :: [*]) (codes :: [[[Atom kon]]]) where
  from_mrec :: SNat ix → El fam ix → Rep_mrec κ (El fam) (Lkup codes ix)
  to_mrec   :: SNat ix → Rep_mrec κ (El fam) (Lkup codes ix) → El fam ix

```

Figure 4. *Family* type class with support for different opaque types

```

geq :: (Family κ fam codes)
    => (∀ k . κ k → κ k → Bool)
    → El fam ix → El fam ix → Bool
geq eq_K x y = go (deepFrom x) (deepFrom y)
  where go (Fix x) (Fix y)
        = maybe False (elimRep (uncurry eq_K) (uncurry go) and)
        $ zipRep x y

```

Figure 5. Generic equality

generics-mrsop we can define a particularly elegant version of generic equality, given in Figure 5.

Reading through the code we see that we convert both arguments of *geq* to their deep representation, then compare their top level constructor with *zipRep*. If they agree we go through each of their fields calling either the equality on opaque types *eq_K* or recursing.

5.2 α -Equivalence

A more involved exercise is the definition of α -equivalence for a language. In this section we start by showing a straightforward version for the λ -calculus and then move on to a more elaborate language. Although such problem has already been treated using generic programming [25], it provides a good example to illustrate our library.

Regardless of the language, determining whether two programs are α -equivalent requires one to focus on the constructors that introduce scoping, declare variables or reference variables. All the other constructors of the language should just combine the recursive results. Let us warm up with untyped λ -calculus:

```
data Term $\lambda$  = Var String | Abs String Term $\lambda$  | App Term $\lambda$  Term $\lambda$ 
```

Let us explain the process step by step. First, for $t_1, t_2 :: \text{Term}_\lambda$ to be α -equivalent, they have to have the constructors on the same positions. Otherwise, they cannot be α -equivalent. Then we check the bound variables: we traverse both terms at the same time and every time we go through a binder, in this case *Abs*, we register a new *rule* saying that the bound variable names are equivalent for the terms under that scope. Whenever we find a reference to a variable, *Var*, we check if the referenced variable is equivalent under the registered *rules* so far.

Let us abstract away this book-keeping functionality by the means of a monad with a couple of associated functions. The idea is that monad m will keep track of a stack of scopes, and each scope will register a list of *name-equivalences*. Indeed, this is very close to how one should go about defining equality for *nominal terms* [3].

```

class Monad m => MonadAlphaEq m where
  scoped :: m a → m a
  addRule :: String → String → m ()
  (≈) :: String → String → m Bool

```

Running a *scoped f* computation will push a new scope for running *f* and pop it after *f* is done. The *addRule v₁ v₂* function registers an equivalence of v_1 and v_2 in the top of the scope stack. Finally, $v_1 \approx v_2$ is defined by pattern matching on the scope stack. If the stack is empty, then $(\approx) v_1 v_2 = (v_1 \equiv v_2)$. Otherwise, let the stack be $s:ss$. We first traverse s gathering the rules referencing either v_1 or v_2 . If there are none, we check if $v_1 \approx v_2$ under ss . If there are rules referencing either variable name in the topmost stack, we must ensure there is only one such rule, and it states a name equivalence between v_1 and v_2 . The implementation of these functions for *MonadAlphaEq* (*State* $[[\text{String}, \text{String}]]$) is available as part of our library.

Returning to our main focus and leaving book-keeping functionality aside, we define in Figure 6 our alpha equivalence decision procedure by encoding what to do for *Var* and *Abs* constructors. The *App* can be eliminated generically.

There is a number of remarks to be made for this example. First, note the application of *zipRep*. If two *Term λ* s are made with different constructors, *galphaEq* will already return *False* because *zipRep* will fail. When *zipRep* succeeds though, we get access to one constructor with paired fields inside. The *go* is then responsible for performing the necessary semantic actions for the *Var* and *Abs* constructors and applying a

```

alphaEq :: Term $\lambda$  → Term $\lambda$  → Bool
alphaEq x y = flip runState [[]]
              (galphaEq (deepFrom x) (deepFrom y))

where
  galphaEq x y = maybe False (go Term) (zipRep x y)
  step = elimRepM (return ∘ uncurry (≈))
        -- opaque types have to be equal!
        (uncurry galphaEq) -- recursive step
        (return ∘ and)    -- combine

go Term $\lambda$  x = case sop x of
  Var (v1 :: v2) → v1 ≈ v2
  Abs (v1 :: v2) (t1 :: t2)
    → scoped (addRule v1 v2 >> galphaEq t1 t2)
  _ → step x

```

Figure 6. α -equivalence for a λ -calculus

data <i>Stmt</i> = <i>SAssign</i> <i>String</i> <i>Exp</i>	$go \overline{Stmt} \ x = \text{case } sop \ x \text{ of}$	
<i>SIf</i> <i>Exp</i> <i>Stmt</i> <i>Stmt</i>	$\overline{SAssign} \ (v_1 :: v_2) \ (e_1 :: e_2)$	$\rightarrow addRule \ v_1 \ v_2 \gg galphaEq \ e_1 \ e_2$
<i>SSeq</i> <i>Stmt</i> <i>Stmt</i>	$-$	$\rightarrow step \ x$
<i>SReturn</i> <i>Exp</i>	$go \overline{Decl} \ x = \text{case } sop \ x \text{ of}$	
<i>SDecl</i> <i>Decl</i>	$\overline{DVar} \ (v_1 :: v_2)$	$\rightarrow addRule \ v_1 \ v_2 \gg return \ True$
<i>SSkip</i>	$\overline{DFun} \ (f_1 :: f_2) \ (x_1 :: x_2) \ (s_1 :: s_2)$	$\rightarrow addRule \ f_1 \ f_2$
data <i>Decl</i> = <i>DVar</i> <i>String</i>	$-$	$\gg scoped \ (addRule \ x_1 \ x_2 \gg galphaEq \ s_1 \ s_2)$
<i>DFun</i> <i>String</i> <i>String</i> <i>Stmt</i>	$-$	$\rightarrow step \ x$
data <i>Exp</i> = <i>EVar</i> <i>String</i>	$go \overline{Exp} \ x = \text{case } sop \ x \text{ of}$	
<i>ECall</i> <i>String</i> <i>Exp</i>	$\overline{EVar} \ (v_1 :: v_2)$	$\rightarrow v_1 \approx v_2$
<i>EAdd</i> <i>Exp</i> <i>Exp</i>	$\overline{ECall} \ (f_1 :: f_2) \ (e_1 :: e_2)$	$\rightarrow (\wedge) \ <\$> \ f_1 \approx f_2 \ <*> \ galphaEq \ e_1 \ e_2$
<i>ESub</i> <i>Exp</i> <i>Exp</i>	$-$	$\rightarrow step \ x$
<i>ELit</i> <i>Int</i>	$go \ - \ x = step \ x$	

Figure 7. α -equivalence for a toy imperative language

general eliminator for anything else. In the actual library, the *pattern synonyms* $\overline{Term}_\lambda$, $\overline{Var}$, and $\overline{Abs}$ are automatically generated as we will see in Appendix B.

One might be inclined to believe that the generic programming here is more cumbersome than a straightforward pattern matching definition over $\overline{Term}_\lambda$. If we consider a more intricate language, however, manual pattern matching becomes almost intractable very fast.

Take the toy imperative language defined in Figure 7. α -equivalence for this language can be defined with just a couple of changes to the definition for $\overline{Term}_\lambda$. For one thing, *alphaEq*, *step* and *galphaEq* remain the same. We just need to adapt the *go* function. Here writing α -equivalence by pattern matching is not straightforward anymore. Moreover, if we decide to change this language and add more statements or more expressions, the changes to the *go* function are minimal, none if we do not introduce any additional construct which declares or uses variables. As long as we do not touch the constructors that *go* patterns matches on, we can even use the very same function.

In this section we have shown several recurring examples from the generic programming community. *generics-mrsop* gives both expressive power and convenience. The last point we have to address is that we still have to write the *Family* instance for the types we want to use. For instance, the *Family* instance for example in Figure 7 is not going to be fun. Deriving these automatically is possible, but non-trivial; we give a full account in Appendix B.

6 Conclusion and Future Work

Generic programming is an ever changing field. The more the Haskell language evolves, the more interesting generic programming libraries we can create. Indeed, some of the language extensions we require in our work were not available at the time that some of the libraries in the related work were developed.

Future work involves expanding the universe of datatypes that our library can handle. Currently, every type involved in a recursive family must be a ground type (of kind $*$ in Haskell terms); our Template Haskell derivations acknowledges this fact by implementing some amount of reduction for types. This limits the functions we can implement generically, for example we cannot write a generic *fmap* function, since it operates on types of kind $* \rightarrow *$. GHC.Generics supports type constructors with exactly one argument via the *Generic1* type class. We intend to combine the approach in this paper with that of Serrano and Miraldo [21], in which atoms have a wider choice of shapes.

The original sum-of-products approach does not handle all the ground types either, only regular ones [4]. We inherit this restriction, and cannot represent recursive families which involve existentials or GADTs. The problem in this case is representing the constraints that each constructor imposes on the type arguments.

Our *generics-mrsop* is a powerful library for generic programming that combines the advantages of previous approaches to generic programming. We have carefully blended the information about (mutually) recursive positions from *multirec*, with the sums-of-products codes introduced by *generics-sop*, while maintaining the advantages of both. The programmer is now able to use simple, combinator-based generic programming for a more expressive class of types than the sums-of-products approach allows. This is interesting, especially since mutually recursive types were hard to handle in a generic fashion previous to *generics-mrsop*.

References

- [1] Michael D. Adams. 2010. Scrap Your Zippers: A Generic Zipper for Heterogeneous Types. In *WGP '10: Proceedings of the 2010 ACM SIGPLAN workshop on Generic programming*. ACM, New York, NY, USA, 13–24. <https://doi.org/10.1145/1863495.1863499>
- [2] Thorsten Altenkirch, Neil Ghani, Peter Hancock, Conor McBride, and Peter Morris. 2015. Indexed containers. *Journal of Functional Programming* 25 (2015).

- [3] Christophe Calvès and Maribel Fernández. 2008. Nominal Matching and Alpha-Equivalence. In *Logic, Language, Information and Computation*, Wilfrid Hodges and Ruy de Queiroz (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 111–122.
- [4] Edsko de Vries and Andres Löb. 2014. True Sums of Products. In *Proceedings of the 10th ACM SIGPLAN Workshop on Generic Programming (WGP '14)*. ACM, New York, NY, USA, 83–94. <https://doi.org/10.1145/2633628.2633634>
- [5] Richard A. Eisenberg and Stephanie Weirich. 2012. Dependently Typed Programming with Singletons. *SIGPLAN Not.* 47, 12 (Sept. 2012), 117–130. <https://doi.org/10.1145/2430532.2364522>
- [6] Richard A. Eisenberg, Stephanie Weirich, and Hamidhasan G. Ahmed. 2016. Visible Type Application. In *Programming Languages and Systems - 25th European Symposium on Programming, ESOP 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings (Lecture Notes in Computer Science)*, Peter Thiemann (Ed.), Vol. 9632. Springer, 229–254.
- [7] Jeremy Gibbons. 2006. Design Patterns As Higher-order Datatype-generic Programs. In *Proceedings of the 2006 ACM SIGPLAN Workshop on Generic Programming (WGP '06)*. ACM, New York, NY, USA, 1–12. <https://doi.org/10.1145/1159861.1159863>
- [8] Ralf Hinze, Johan Jeuring, and Andres Löb. 2004. Type-indexed data types. *Science of Computer Programming* 51, 1 (2004), 117 – 151. <https://doi.org/10.1016/j.scico.2003.07.001> Mathematics of Program Construction (MPC 2002).
- [9] Gérard Huet. 1997. The Zipper. *Journal of Functional Programming* 7, 5 (1997), 549–554.
- [10] Ralf Lämmel and Simon Peyton Jones. 2003. Scrap Your Boilerplate: A Practical Design Pattern for Generic Programming. In *Proceedings of the 2003 ACM SIGPLAN International Workshop on Types in Languages Design and Implementation (TLDI '03)*. ACM, New York, NY, USA, 26–37. <https://doi.org/10.1145/604174.604179>
- [11] Andres Löb and José Pedro Magalhães. 2011. Generic programming with indexed functors. In *Proceedings of the seventh ACM SIGPLAN workshop on Generic programming*. ACM, 1–12.
- [12] José Pedro Magalhães, Atze Dijkstra, Johan Jeuring, and Andres Löb. 2010. A Generic Deriving Mechanism for Haskell. In *Proceedings of the Third ACM Haskell Symposium on Haskell (Haskell '10)*. ACM, New York, NY, USA, 37–48. <https://doi.org/10.1145/1863523.1863529>
- [13] José Pedro Magalhães and Andres Löb. 2012. A Formal Comparison of Approaches to Datatype-Generic Programming. In *Proceedings Fourth Workshop on Mathematically Structured Functional Programming, Tallinn, Estonia, 25 March 2012 (Electronic Proceedings in Theoretical Computer Science)*, James Chapman and Paul Blain Levy (Eds.), Vol. 76. Open Publishing Association, 50–67. <https://doi.org/10.4204/EPTCS.76.6>
- [14] Simon Marlow et al. 2010. Haskell 2010 Language Report. <https://www.haskell.org/onlinereport/haskell2010/>.
- [15] Victor Cacciari Miraldo, Pierre-Évariste Dagand, and Wouter Swierstra. 2017. Type-directed Diffing of Structured Data. In *Proceedings of the 2Nd ACM SIGPLAN International Workshop on Type-Driven Development (TyDe 2017)*. ACM, New York, NY, USA, 2–15. <https://doi.org/10.1145/3122975.3122976>
- [16] Neil Mitchell and Colin Runciman. 2007. Uniform Boilerplate and List Processing. In *Proceedings of the ACM SIGPLAN Workshop on Haskell Workshop (Haskell '07)*. ACM, New York, NY, USA, 49–60. <https://doi.org/10.1145/1291201.1291208>
- [17] Thomas van Noort, Alexey Rodriguez, Stefan Holdermans, Johan Jeuring, and Bastiaan Heeren. 2008. A Lightweight Approach to Datatype-generic Rewriting. In *Proceedings of the ACM SIGPLAN Workshop on Generic Programming (WGP '08)*. ACM, New York, NY, USA, 13–24. <https://doi.org/10.1145/1411318.1411321>
- [18] Matthew Pickering, Gergő Erdi, Simon Peyton Jones, and Richard A. Eisenberg. 2016. Pattern Synonyms. In *Proceedings of the 9th International Symposium on Haskell (Haskell 2016)*. ACM, New York, NY, USA, 80–91. <https://doi.org/10.1145/2976002.2976013>
- [19] Alexey Rodriguez, Johan Jeuring, Patrik Jansson, Alex Gerdes, Oleg Kiselyov, and Bruno C. d. S. Oliveira. 2008. Comparing Libraries for Generic Programming in Haskell. In *Proceedings of the First ACM SIGPLAN Symposium on Haskell (Haskell '08)*. ACM, New York, NY, USA, 111–122. <https://doi.org/10.1145/1411286.1411301>
- [20] Alejandro Serrano and Jurriaan Hage. 2016. Generic Matching of Tree Regular Expressions over Haskell Data Types. In *Practical Aspects of Declarative Languages - 18th International Symposium, PADL 2016, St. Petersburg, FL, USA, January 18-19, 2016. Proceedings*. 83–98. https://doi.org/10.1007/978-3-319-28228-2_6
- [21] Alejandro Serrano and Victor Cacciari Miraldo. 2018. Generic Programming of All Kinds. In *Conditionally accepted to Haskell Symposium 2018 (Haskell '18)*.
- [22] Tim Sheard and Simon Peyton Jones. 2002. Template meta-programming for Haskell. 1–16. <https://www.microsoft.com/en-us/research/publication/template-meta-programming-for-haskell/>
- [23] Stephanie Weirich, Justin Hsu, and Richard A. Eisenberg. 2013. System FC with Explicit Kind Equality. *SIGPLAN Not.* 48, 9 (Sept. 2013), 275–286. <https://doi.org/10.1145/2544174.2500599>
- [24] Stephanie Weirich, Antoine Voizard, Pedro Henrique Azevedo de Amorim, and Richard A. Eisenberg. 2017. A Specification for Dependent Types in Haskell. *Proc. ACM Program. Lang.* 1, ICFP, Article 31 (Aug. 2017), 29 pages. <https://doi.org/10.1145/3110275>
- [25] Stephanie Weirich, Brent A. Yorgey, and Tim Sheard. 2011. Binders Unbound. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming (ICFP '11)*. ACM, New York, NY, USA, 333–345. <https://doi.org/10.1145/2034773.2034818>
- [26] Hongwei Xi, Chiyen Chen, and Gang Chen. 2003. Guarded Recursive Datatype Constructors. In *Proceedings of the 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '03)*. ACM, New York, NY, USA, 224–235. <https://doi.org/10.1145/604131.604150>
- [27] Alexey Rodriguez Yakushev, Stefan Holdermans, Andres Löb, and Johan Jeuring. 2009. Generic Programming with Fixed Points for Mutually Recursive Datatypes. In *Proceedings of the 14th ACM SIGPLAN International Conference on Functional Programming (ICFP '09)*. ACM, New York, NY, USA, 233–244. <https://doi.org/10.1145/1596550.1596585>
- [28] Brent A. Yorgey, Stephanie Weirich, Julien Cretin, Simon Peyton Jones, Dimitrios Vytiniotis, and José Pedro Magalhães. 2012. Giving Haskell a Promotion. In *Proceedings of the 8th ACM SIGPLAN Workshop on Types in Language Design and Implementation (TLDI '12)*. ACM, New York, NY, USA, 53–66. <https://doi.org/10.1145/2103786.2103795>